Dispensa: I database NoSQL

Francesco Antonio Mangiacotti, Emanuele Steduto, Alberto D'Addetta

1 Il concetto di database NoSQL

I database NoSQL, spesso chiamati "Not only SQL" o non relazionali, rappresentano un approccio all'archiviazione dei dati che si discosta dai sistemi tradizionali basati su tabelle e relazioni rigide.

2 Le caratteristiche di un database NoSQL

- **Flessibilità dello schema:** A differenza dei database SQL che richiedono una struttura predefinita (schema), i database NoSQL hanno spesso un design schema-less. Questo permette di archiviare dati senza dover definire tabelle fisse in anticipo, rendendoli ideali per gestire set di dati non strutturati o situazioni in cui la struttura dei dati evolve rapidamente.
- **Scalabilità orizzontale:** Uno dei principali vantaggi è la capacità di scalare orizzontalmente aggiungendo server comuni a un cluster, invece di dover potenziare un singolo server (scalabilità verticale). Questo avviene spesso tramite lo *sharding*, ovvero la distribuzione automatica dei dati su più nodi, facilitando la gestione di traffico elevato e dataset massicci.
- **Agilità di sviluppo:** Poiché non richiedono un complesso lavoro preliminare per definire le relazioni tra i dati, permettono agli sviluppatori di iniziare i progetti più rapidamente e di adattare il database man mano che le esigenze dell'applicazione cambiano.
- **Gestione delle relazioni:** Nei sistemi NoSQL non esistono le relazioni fisse e i join complessi tipici di SQL. Se è necessario combinare dati provenienti da fonti diverse, questa logica deve essere solitamente gestita a livello di codice applicativo.

3 Le proprietà BASE

Modello di coerenza BASE: Mentre i database tradizionali puntano alla coerenza immediata (ACID), molti NoSQL adottano il modello BASE. Questo approccio privilegia la disponibilità del sistema, accettando che i dati potrebbero non essere istantaneamente sincronizzati su tutte le repliche, ma convergeranno verso uno stato coerente nel tempo.

Acronimo BASE:

- **Basically Available (sostanzialmente disponibile):** Questa proprietà indica che il sistema dà priorità alla disponibilità per le operazioni di lettura e scrittura. Anche in caso di guasti a singoli nodi o di partizioni di rete (problemi di comunicazione tra i server), il database cercherà comunque di rispondere alle richieste degli utenti invece di bloccarsi per garantire la perfezione del dato.

- **Soft State (Stato Morbido/Variabile):** Si riferisce al fatto che lo stato del sistema può cambiare nel tempo, anche senza che arrivino nuovi input esterni. Poiché i dati non vengono sincronizzati istantaneamente ovunque, lo stato di un nodo potrebbe essere temporaneamente non aggiornato mentre attende di ricevere le ultime modifiche dagli altri nodi.
- **Eventual Consistency (Coerenza Finale):** È il concetto chiave del modello. Significa che le modifiche ai dati potrebbero non essere riflesse istantaneamente su tutte le repliche o copie del database. Tuttavia, se non vengono effettuate nuove modifiche, il sistema convergerà gradualmente e alla fine tutti i nodi saranno allineati allo stesso stato coerente.

In sintesi, il modello BASE accetta il rischio che un utente possa leggere un dato leggermente "vecchio" per un breve istante, in cambio di una capacità di gestione di volumi di traffico e dati massicci che sarebbe impossibile ottenere con una coerenza rigorosa e immediata.

4 I 4 modelli no sql maggiormente usati

- **Key-Value Stores (Archivi Chiave-Valore):** È il modello dove i dati sono memorizzati come coppie composte da una chiave univoca e un valore associato, funzionando come una grande mappa hash. Questo approccio garantisce massima velocità e semplicità, rendendolo ideale per il caching e la gestione delle sessioni. Esistono diversi tipi di databases basati sul modello di archiviazione chiave valore:
  - **Database chiave-valore in-memory:** Sono ottimizzati per il caching e le prestazioni estreme, poiché i dati risiedono principalmente nella RAM (memoria volatile). Esempi comuni includono Redis e memCached. Utilizzo: Sono ideali per il caching*, la gestione delle sessioni utente e l'accesso casuale ai dati in tempo reale (come nel settore del gaming).

    *caching = salvare temporaneamente il risultato di un'operazione costosa per non rifarlo ogni volta. Esempio di caching:

```
senza cache:
Utente richiede pagina -> PHP interroga MySQL -> MySQL legge disco ->
    risposta in 300ms
Utente richiede stessa pagina -> PHP interroga MySQL -> MySQL legge
    disco -> 300ms ancora

con cache:
Prima richiesta -> PHP interroga MySQL -> salva risultato in RAM -> 300
    ms
Seconda richiesta -> PHP legge da RAM -> risposta in 2ms
```

Listing 1: Esempio di Caching

  - **Database chiave-valore persistenti:** Questi sistemi archiviano i dati su disco (storage persistente) per garantirne la conservazione anche in caso di riavvio del sistema. MongoDB e Redis (se configurato per la persistenza) rientrano in questa categoria. Vengono utilizzati quando la velocità di accesso deve essere combinata con la sicurezza della conservazione a lungo termine.
  - **Database chiave-valore multi-modello:** Sono database flessibili che supportano simultaneamente più modelli di dati, come documenti, grafi e chiavi-valore, all'interno della stessa istanza. MongoDB e CosmosDB sono esempi di questa tipologia. Sono ideali per software che richiedono più di una semplice ricerca per chiave, permettendo di combinare la velocità del modello chiave-valore con funzionalità avanzate.

In sintesi, i sistemi in-memory puntano alla velocità pura, quelli persistenti alla sicurezza del dato, mentre i multi-modello offrono versatilità per gestire diverse tipologie di informazioni nello stesso sistema.

- **Document Databases (Database a Documenti):** Archiviano i dati in documenti in formato JSON o BSON, permettendo di inserire strutture complesse e nidificate all'interno di un singolo record. Poiché non esiste uno schema rigido, i campi possono variare da un documento all'altro, offrendo grande flessibilità nello sviluppo (es. MongoDB) Esempi di JSON di un database che utilizza questo modello:

```
{
    "_id": 1,
    "nome": "Tommaso",
    "email": "tommaso@example.com",
    "cellulare": "765-555-5555",
    "interessi": [
        "moda",
        "spa",
        "shopping"
    ],
    "aziende": [
        {
            "nome": "Entertainment 1080",
            "socio": "Giovanni",
            "stato": "Fallita",
            "data_fondazione": {
                "$date": "2012-05-19T04:00:00Z"
            }
        },
        {
            "nome": "Swag for Tweens",
            "data_fondazione": {
                "$date": "2012-11-01T04:00:00Z"
            }
        }
    ]
}
```

```
{
    "_id": 2,
    "nome": "Donna",
    "email": "donna@example.com",
    "coniuge": "Joe",
    "interessi": [
        "spa",
        "shopping",
        "live tweeting"
    ],
    "aziende": [
        {
            "nome": "Castle Realty",
            "stato": "Fiorente",
            "data_fondazione": {
                "$date": "2013-11-21T04:00:00Z"
            }
        }
    ]
}
```

Figura 1: Esempio di Document Database (JSON)

Si può notare che il documento relativo a Donna non contiene gli stessi campi del documento relativo a Tommaso. La raccolta dati degli utenti utilizza uno schema flessibile per memorizzare le informazioni relative a ciascun utente. Utilizzi nel mondo reale:

1. **Piattaforme di E-commerce e Retail:** Sono ideali per la gestione dei cataloghi prodotti, dove ogni articolo può avere attributi diversi (es. taglie per i vestiti, specifiche tecniche per l'elettronica) che non si adattano bene a una tabella rigida.
2. **Sistemi di Gestione dei Contenuti (CMS):** Sono utilizzati per archiviare blog, articoli, video e metadati correlati che evolvono nel tempo. La flessibilità dello schema permette di aggiungere nuovi campi (come nuovi tag o tipi di allegati) senza dover modificare l'intera struttura del database.
3. Qualunque applicazione che richiede elevata flessibilità nello schema dei dati, scalabilità orizzontale e la possibilità di gestire strutture dati eterogenee o in continua evoluzione.

- **Wide-Column Stores (archivi a colonne larghe):** Utilizzano una struttura simile alle tabelle dei database relazionali, composta da righe e colonne. Tuttavia, a differenza dei database SQL — dove le colonne sono fisse e uguali per tutte le righe — qui ogni riga può contenere un numero e un tipo di colonne completamente diversi, rendendo la struttura più simile a una mappa di mappe che a una tabella tradizionale. I dati sono organizzati in famiglie di colonne, ovvero gruppi logici di colonne definite a livello di schema, all'interno delle quali ogni riga può contenere un sottoinsieme diverso di colonne. A differenza dei database SQL, che salvano i dati riga per riga, i Wide-Column Store salvano i dati fisicamente per colonna: questo rende le query analitiche molto più efficienti

(es. media di tutti i prezzi), poiché è possibile leggere solo la colonna di interesse senza caricare l'intera riga. Questo modello consente un'elevata scalabilità su molti nodi ed è particolarmente adatto alla gestione di grandi volumi di dati, operazioni di scrittura intensive e interrogazioni su singole colonne su milioni di record. Esempio generico di uno schema di un database che usa questo modello:

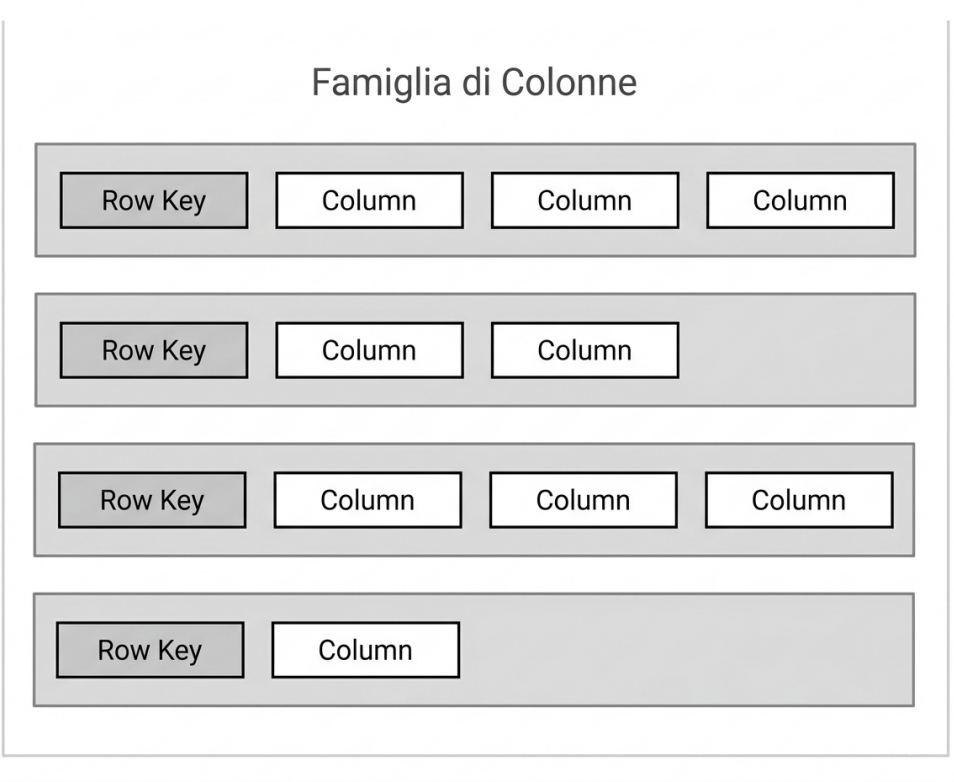

Figura 2: Esempio generico di schema Wide-Column Store (Famiglia di Colonne)

Esempi di utilizzo nel mondo reale dei wide-column stores:

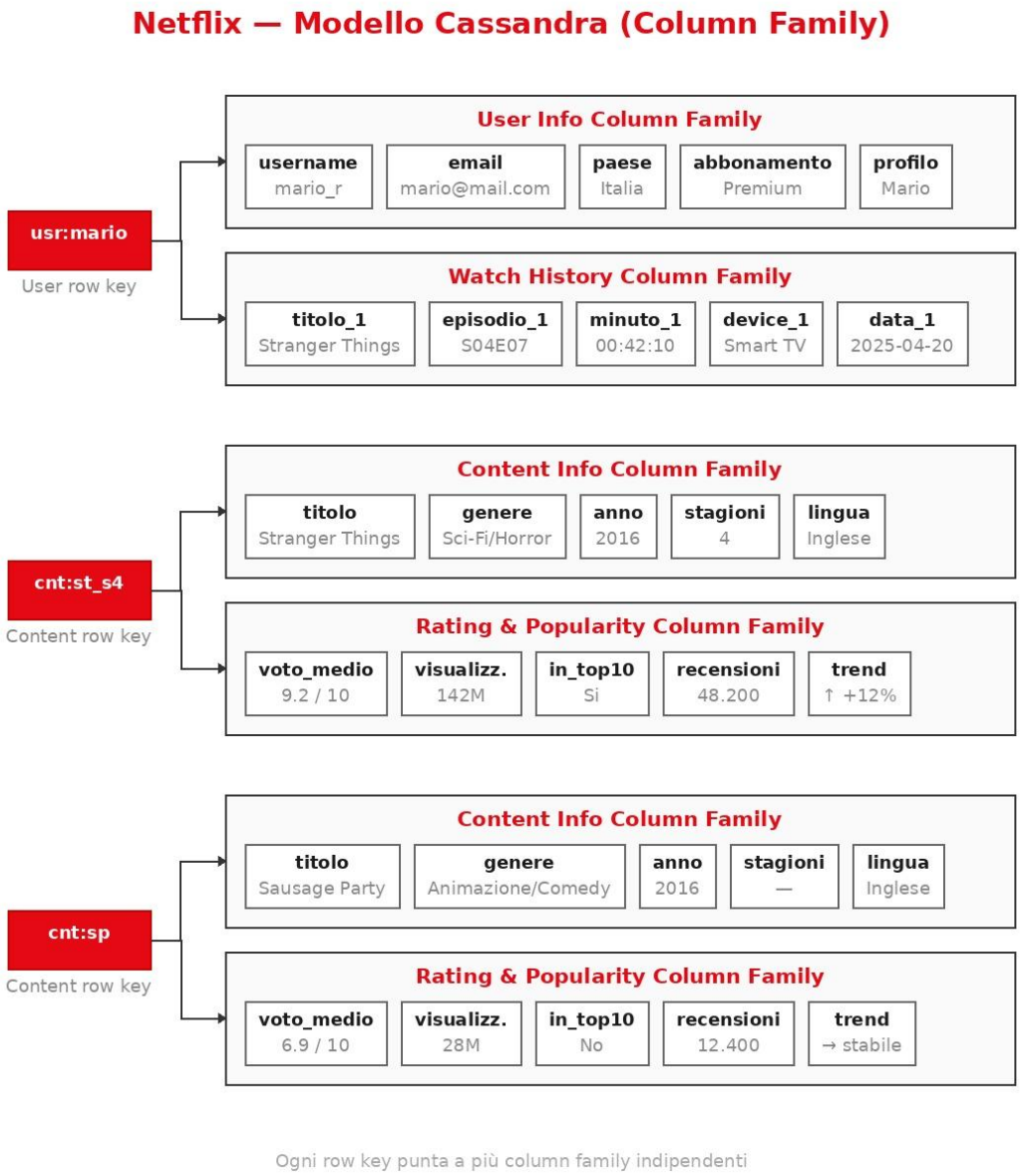

Figura 3: Esempio di Wide-Column Store (Cassandra) per Netflix

Ogni colonna presente in una riga (es. username, email, paese, abbonamento, profilo, nell'insieme di colonne "user info column family", indipendente l'una dall'altra. In SQL una riga è un blocco unico, se vuoi leggere solo l'email, il database carica comunque tutta la riga (username, paese, abbonamento, ecc.). In Cassandra ogni colonna è salvata fisicamente separata. Puoi leggere solo email senza toccare le altre. E soprattutto, ogni riga può avere colonne diverse: si può notare con la colonna stagioni presente nella "content info column family" di Stranger Things è 4, mentre la stessa colonna di Sausage Party è —, ciò non significa che è null, ma proprio che fisicamente non esiste quella colonna. App reali che usano questo modello:

1. **Spotify** Archivia la cronologia di ascolto e le playlist di oltre 600 milioni di utenti.
2. **Netflix** Gestisce i progressi di visione di milioni di utenti simultaneamente. Ogni volta che metti in pausa un film, Cassandra salva il punto esatto in tempo reale su scala globale.

3. **Instagram** Usato per salvare le attività degli utenti: like, commenti, notifiche. Gestisce miliardi di scritture al giorno.

Denominatore comune: tutti questi servizi hanno in comune scritture massicce, scala globale e nessuna tolleranza ai downtime, esattamente il punto di forza del modello wide-column store.

- **Graph Databases (Database a Grafo):** Si concentrano sulle relazioni tra entità, memorizzandole come nodi ed archi. Sono lo strumento d'elezione per analizzare connessioni complesse, come quelle dei social network o dei sistemi di raccomandazione prodotti. Per spiegare bene e semplicemente i graph databases, bisogna partire dal problema dei database SQL tradizionali: In un database SQL, per collegare due tabelle (es. Utenti e Amici), il sistema deve usare degli **indici**. Se io chiedessi, "chi sono gli amici di Pietro?"il db seguirebbe questi passaggi:

  1. Il DB cerca "Pietro" nell'indice degli utenti.
  2. Poi deve cercare nella tabella dei legami tutti i record che corrispondono all'ID di Pietro.
  3. Più la tabella dei legami diventa grande (milioni o miliardi di righe), più tempo impiega l'indice per trovare i risultati.

  Risultato: lentezza, complessità e inefficienza.

  Soluzione tramite i database a grafo: puntatori fisici

  In un database a grafo (come Neo4j), i dati non sono tabelle separate, ma oggetti interconnessi in memoria. Ogni "Nodo" (es. l'utente Pietro) contiene al suo interno una lista di **puntatori diretti, i cosiddetti archi** (indirizzi fisici sul disco/RAM) ai nodi a cui è collegato. Quando sei sul nodo "Pietro", non c'è bisogno di fare una ricerca globale per trovare gli amici, ma semplicemente seguire i fili (puntatori) che possiede per fare la ricerca. Questo approccio ha svariati pro, infatti la velocità con cui troverà gli amici sarà costante, sia che il db avrà 100000 nodi o 100 nodi, perché i passaggi che farà saranno sempre gli stessi, infatti si parla di ricerca locale, non globale. In sintesi, la performance dei Graph Database non è influenzata dalla dimensione totale del dataset (Big Data), ma dalla **dimensione del sottografo** che si sta esplorando (es: se Pietro ha 1 milione di amici è ovvio che la velocità con cui si troverà il risultato sarà minore rispetto al caso in cui Pietro abbia 100 amici).

  Esempi di utilizzo di questo modello nel mondo reale:

  1. **Social network:** suggerimenti di amicizia su instagram, facebook, tiktok, ecc. Pietro (Nodo) AMICO DI (Arco/Puntatore) Sara (Nodo)
     Pietro (Nodo) AMICO DI (Arco/Puntatore) Luca (Nodo)
     Sara (Nodo) AMICO DI (Arco/Puntatore) Anna (Nodo)
     Risultato: Anna verrà suggerita a Pietro tra le possibili amicizie.
  2. **Motori di raccomandazione:** Se l'utente A ha visto il film X e l'utente B (molto simile ad A per interessi) ha visto X e Y, il sistema suggerisce Y all'utente A seguendo i percorsi nel grafo, stesso ragionamento anche per suggerimenti di acquisto prodotti.
  3. **AI:** i grafi danno a disposizione alle AI informazioni vere su cui basarsi, così da non permettere a queste ultime di dire cose non vere o senza senso (allucinazioni). In più forniscono all'AI una base di dati su cui ragionare, ad esempio Parigi (città) che è collegata con Francia (nazione) e con Torre Eiffel (monumento), permettendo all'AI di fare collegamenti complessi che non riuscirebbe a fare analizzando solamente del testo. Quest'approccio permette anche di capire perché l'IA ha preso una specifica decisione,

seguendo i passaggi attraverso il grafo che ha fatto, rendendo così più trasparente il processo decisionale.

5 Il teorema cap

Il Teorema CAP è un principio fondamentale dei sistemi distribuiti che afferma che è impossibile per un database distribuito fornire simultaneamente più di due delle seguenti tre garanzie:

- **Consistency (Coerenza):** Ogni lettura riceve la scrittura più recente o un errore. In un sistema coerente, tutti i nodi vedono gli stessi dati nello stesso momento.
- **Availability (Disponibilità):** Ogni richiesta riceve una risposta (che non sia un errore), ma senza la garanzia che contenga l'informazione più recente. Il sistema deve rimanere operativo e rispondere alle richieste anche in caso di guasti parziali.
- **Partition Tolerance (Tolleranza alla partizione):** Il sistema continua a funzionare nonostante una comunicazione interrotta o messaggi persi tra i nodi della rete.

Poiché i database NoSQL sono progettati per essere distribuiti su più server, la tolleranza alla partizione è considerata una necessità imprescindibile, dato che i problemi di rete in cluster di grandi dimensioni sono inevitabili. Di conseguenza, la scelta ricade solitamente tra Coerenza o Disponibilità:

1. **Sistemi CP (Coerenza e Tolleranza alla partizione):** Se si verifica un problema di rete che impedisce ai nodi di sincronizzarsi, il sistema sceglierà di non rispondere a determinate richieste per non rischiare di fornire dati incoerenti o non aggiornati. Esempio: MongoDB è spesso classificato come un sistema CP. Utilizza un modello a "singolo Master" (Primary node). Tutte le scritture vanno al Primary, che poi le replica ai Secondary. Se il nodo Primary si disconnette, il sistema ferma le scritture e avvia un'elezione per scegliere un nuovo leader. Durante questo tempo (alcuni secondi), il sistema non è disponibile per la scrittura, garantendo però che nessuno scriva dati che potrebbero andare persi o creare conflitti.
2. **Sistemi AP (Disponibilità e Tolleranza alla partizione):** Il sistema darà priorità alla risposta all'utente. In caso di partizione di rete, i nodi continueranno a servire i dati anche se questi potrebbero non essere ancora perfettamente sincronizzati tra loro. Esempio: Apache Cassandra, usa un'architettura "Peer-to-Peer" senza un master. Ogni nodo può ricevere letture e scritture. Se un nodo non riesce a parlare con gli altri, continua comunque ad accettare dati dagli utenti. Quando la rete torna a funzionare, i nodi si scambiano le novità e si riallineano.

In sintesi, si tende a scegliere un sistema CP quando l'integrità dei dati è un elemento fondamentale e non puoi permetterti che l'utente legga dati sbagliati o obsoleti (es. settore finanziario). Si mira ad usare invece un sistema AP quando la bassa latenza e la presenza di dati costante è più importante della sincronizzazione dei dati tra tutti i nodi (es. social network).

6 Quando usare noSQL rispetto a un database relazionale tradizionale

La scelta tra un database NoSQL e uno relazionale (SQL) dipende dalle necessità specifiche del progetto, dal tipo di dati gestiti e dalle previsioni di crescita dell'applicazione. Le aziende oggi spesso non scelgono l'uno a scapito dell'altro, ma utilizzano entrambi in architetture ibride per sfruttare i punti di forza di ciascuno. Vediamo però i casi in cui le aziende scelgono di usare l'uno, l'altro o tutt'e due.

Quando un azienda sceglie NoSQL

Le aziende optano per soluzioni NoSQL quando i requisiti principali sono la velocità di sviluppo, la flessibilità e la capacità di gestire volumi massicci di dati.

- **Scalabilità Orizzontale:** Se un'azienda prevede una crescita esplosiva del traffico o dei dati, i database NoSQL sono ideali perché permettono di aggiungere facilmente server economici al cluster (sharding). Al contrario, scalare un database SQL oltre certi limiti richiede hardware molto costoso e complesso (scalabilità verticale).
- **Dati non strutturati o in rapida evoluzione:** In scenari come lo sviluppo di app mobile o sistemi di gestione contenuti (CMS), dove i requisiti cambiano spesso, la natura schema-less del NoSQL permette di aggiungere nuovi campi ai dati senza dover migrare intere tabelle o interrompere il servizio.
- **Prestazioni di lettura per "Aggregati":** Quando i dati che vengono consultati insieme devono essere memorizzati insieme (es. un post con tutti i suoi commenti o un corso con le sue lezioni), il NoSQL permette di recuperarli in un'unica operazione, evitando i costosi join dei database SQL.

Quando un'azienda preferisce un database Relazionale (SQL)

I database SQL rimangono la scelta preferita quando l'integrità dei dati e la complessità delle relazioni sono prioritarie rispetto alla scalabilità estrema.

- **Transazioni ACID e Affidabilità:** In ambiti come il banking, la finanza o la contabilità, è fondamentale che ogni transazione sia atomica, coerente, isolata e duratura. Il modello SQL garantisce che un errore di sistema non lasci mai i dati in uno stato incoerente (es. soldi spariti da un conto ma non ancora accreditati su un altro).
- **Schema Rigido e Prevedibile:** Se i dati hanno una struttura fissa e ben definita che non cambia nel tempo, uno schema rigido aiuta a mantenere la pulizia e la qualità delle informazioni.
- **Query Complesse e Reportistica:** Se l'azienda ha bisogno di eseguire analisi sofisticate incrociando dati provenienti da molte fonti diverse, i join di SQL sono estremamente potenti e ottimizzati per questo scopo.
- **Standardizzazione:** SQL è un linguaggio standardizzato da decenni, il che rende più semplice trovare professionisti competenti e integrare strumenti di business intelligence esistenti.

Lo scenario reale: l'approccio ibrido

Nella realtà, come abbiamo già detto, quasi sempre si utilizza un approccio ibrido, così da poter sfruttare i punti di forza di tutti e due i modelli (noSQL, SQL). Ci sono svariati esempi:

1. **Piattaforme di streaming (netflix o disney+):**
    - **SQL:** Gestisce i profili utenti e abbonamenti. Dati personali, scadenze dei pagamenti e credenziali di accesso. Sono dati che cambiano poco ma devono essere sicuri e consistenti.
    - **NoSQL - Wide Column (Cassandra):** Gestisce la cronologia di visione (Watch History). Ogni volta che metti in pausa o guardi un secondo di video, viene generato un dato. Cassandra permette di scrivere migliaia di questi eventi al secondo senza rallentare.

- **NoSQL - Graph (Neo4j):** Gestisce il motore di raccomandazione. Analizza le relazioni tra utenti ("Chi ha visto questo, ha visto anche...") e le caratteristiche dei film per suggerirti cosa guardare dopo.

2. **Social network (instagram o x):**

   - **SQL:** Gestisce le informazioni di base dell'account. Nome utente, password, impostazioni sulla privacy e dati di profilo.
   - **NoSQL - Graph:** Gestisce il social graph. Chi segue chi, i "like" e le interazioni. Calcolare se sei un "amico di un amico" in SQL richiederebbe JOIN troppo pesanti.
   - **NoSQL - Wide Column:** Gestisce il feed. I post degli utenti che segui vengono "aggregati" e serviti velocemente tramite un database ottimizzato per le letture sequenziali.

Dispensa: I database NoSQL - Parte 2

1 Analisi del modello DB Cassandra Apache

Apache Cassandra è un database NoSQL di tipo "column-family" che presenta una curva di apprendimento media, soprattutto per studenti che hanno già familiarità con i database relazionali tradizionali come MySQL. A differenza di strumenti con curva bassa (come Python o MongoDB), dove scrivi due righe e tutto funziona subito, Cassandra richiede di capire la teoria prima di scrivere il codice. Se provi a usare Cassandra come se fosse MySQL (facendo tabelle normalizzate e cercando di collegarle), il database smetterà di funzionare o diventerà lentissimo non appena i dati aumentano.

Per uno studente che ha già studiato MySQL, la difficoltà iniziale è alta e psicologica: bisogna disimparare per imparare.

- In SQL: "Prima creo le tabelle, poi decido come interrogarle".
- In Cassandra: "Prima decido quali domande farà la mia App, poi creo tabelle specifiche per ogni domanda". Questo ribaltamento logico è ciò che alza l'asticella della difficoltà.

A questa principale difficoltà se ne aggiungono svariate, una tra le più importanti è la gestione degli errori, in un database semplice, se sbagli una query ricevi un errore chiaro. In Cassandra, se sbagli la progettazione della Partition Key (la bussola che dice il dato A andrà nel server N), il database non ti dà errore, ma distribuisce i dati male tra i computer (nodi), creando colli di bottiglia che sono difficili da diagnosticare per un principiante.

A prima vista, Cassandra può sembrare semplice grazie all'utilizzo del linguaggio CQL (Cassandra Query Language), che è progettato per essere simile al linguaggio SQL. Questo rende le operazioni di base, come la creazione di tabelle o l'inserimento di dati, relativamente intuitive per chi ha già studiato SQL.

Confronto tra CQL (Cassandra) e SQL (MySQL)

CQL — Cassandra Query Language

```
CREATE TABLE libri_per_autore (
    autore text,
    titolo text,
    isbn   text,
    anno   int,
    PRIMARY KEY (autore, titolo)
);

INSERT INTO libri_per_autore
    (autore, titolo, isbn, anno)
VALUES (
    'Dante Alighieri',
    'Divina Commedia',
    '978-88',
    1320
);

SELECT * FROM libri_per_autore
WHERE autore = 'Dante Alighieri';
```

- Chiave primaria composta: (autore, titolo)
- Tipi primitivi senza lunghezza: text, int
- La partition key (autore) determina la distribuzione dei dati nel cluster
- La WHERE deve filtrare sulla partition key

SQL — Structured Query Language

```
CREATE TABLE libri (
    isbn   VARCHAR(20) PRIMARY KEY,
    titolo VARCHAR(100),
    autore VARCHAR(100),
    anno   INT
);

INSERT INTO libri
    (isbn, titolo, autore, anno)
VALUES (
    '978-88',
    'Divina Commedia',
    'Dante Alighieri',
    1320
);

SELECT * FROM libri
WHERE autore = 'Dante Alighieri';
```

- Chiave primaria singola: isbn
- Tipi con lunghezza esplicita: VARCHAR(n), INT
- Supporto a vincoli referenziali (FOREIGN KEY)
- La WHERE puo' filtrare su qualsiasi colonna (con o senza indice)

Tabella 1 — Confronto sintattico tra CQL e SQL su operazioni CREATE, INSERT e SELECT

Figura 1: Confronto sintattico tra CQL e SQL su operazioni CREATE, INSERT e SELECT

Nell'esempio mostrato si può notare come i due linguaggi siano molto simili per sintassi, ma differiscono per diversi aspetti fondamentali. In SQL la chiave primaria è tipicamente singola e basata su un identificatore univoco (nel caso mostrato, isbn), mentre in CQL la chiave primaria può essere composta da una partition key e una clustering key, che determinano rispettivamente la distribuzione dei dati nel cluster e l'ordinamento all'interno della partizione. Inoltre, SQL utilizza tipi di dato con lunghezza esplicita come VARCHAR(n), mentre CQL adotta tipi primitivi senza vincoli di lunghezza come text. Un'ulteriore differenza riguarda le query di selezione: in SQL è possibile filtrare su qualsiasi colonna, anche senza un indice, mentre in CQL la clausola WHERE deve obbligatoriamente includere la partition key, poiché il sistema non supporta scansioni complete della tabella in modo efficiente.

Tuttavia, la vera difficoltà non risiede nel linguaggio di interrogazione, ma nel modo in cui si progettano e si organizzano i dati. A differenza dei database relazionali, Cassandra non supporta operazioni come le join tra tabelle, e questo obbliga lo sviluppatore a progettare il database in modo completamente diverso. In particolare, si adotta un approccio denormalizzato, in cui i dati vengono spesso duplicati in più tabelle per ottimizzare le prestazioni delle query. Questo richiede una pianificazione attenta e una buona comprensione delle operazioni che l'applicazione dovrà eseguire.

Inoltre, Cassandra è progettato per essere un database distribuito, cioè pensato per funzionare su più macchine contemporaneamente. Questo introduce concetti aggiuntivi, come la gestione dei nodi, la replicazione dei dati e la consistenza distribuita, che possono risultare complessi per uno studente alle prime armi. Nonostante ciò, questi aspetti rappresentano anche un'opportunità di apprendimento importante, in quanto permettono di comprendere meglio il funzionamento dei sistemi moderni su larga scala.

In conclusione, Cassandra può essere considerato un database impegnativo ma formativo: inizialmente può risultare difficile da comprendere, ma offre competenze molto utili e attuali nel mondo dell'informatica.

2 Infrastrutture e costi

Dal punto di vista dell'infrastruttura, Cassandra richiede risorse hardware non trascurabili: necessita obbligatoriamente di Java e di una configurazione iniziale non immediata. I computer del laboratorio scolastico, equipaggiati con processori Intel Core i5 di undicesima o dodicesima generazione e 16 GB di RAM DDR4, sono tecnicamente sufficienti per avviare un'istanza locale di Cassandra. Tuttavia, il problema principale non è la potenza hardware, bensì la gestione dei permessi: sebbene sia possibile installare il software, Cassandra tende a richiedere privilegi elevati anche durante il normale utilizzo, ad esempio per la gestione dei servizi di sistema. In un ambiente scolastico dove i permessi di amministratore non sono disponibili in modo continuativo, questo rappresenta un ostacolo concreto all'utilizzo quotidiano.

Inoltre, Cassandra esprime il suo pieno potenziale in un contesto distribuito, con più nodi collegati tra loro, una configurazione difficilmente replicabile in laboratorio.

Una soluzione pratica è rappresentata dalle piattaforme cloud come Astra DB di DataStax, che permettono di utilizzare Cassandra direttamente dal browser senza installare nulla in locale e senza necessità di permessi di amministratore. Il piano gratuito offre 25 dollari di crediti mensili (300 dollari/anno), sufficienti a coprire fino a 80 GB di storage e 20 milioni di operazioni al mese, una soglia ampiamente sufficiente per qualsiasi progetto scolastico. I crediti si azzerano ogni mese e i database inattivi vengono messi in pausa automaticamente. In caso vengano consumati tutti i crediti (altamente improbabile per un utilizzo didattico), i database vengono ibernati fino al ripristino dei crediti.

2.1 Differenze nei maggiori OS

Va inoltre considerato che l'installazione locale di Cassandra su Windows, sistema operativo tipicamente adottato nei laboratori scolastici, risulta particolarmente problematica: il database non è nativamente supportato su Windows e richiede l'utilizzo di WSL o Docker, strumenti che a loro volta necessitano di privilegi di amministratore e di risorse aggiuntive. Questo rende la soluzione cloud ancora più vantaggiosa in un contesto scolastico con macchine Windows.

Cloud (Astra DB) — identico su tutti i sistemi. Essendo accessibile dal browser, non cambia nulla tra Windows, Linux e macOS. L'unico requisito è una connessione internet.

In locale:

- Linux: installazione più fluida, Cassandra gira nativamente come servizio di sistema. È l'ambiente per cui Cassandra è progettato.
- macOS: funziona bene, installazione tramite Homebrew o manuale. Qualche differenza nella gestione dei servizi ma niente di problematico.
- Windows: è il caso più critico. Cassandra non ha un supporto nativo ottimale per Windows — ufficialmente è supportato solo tramite WSL (Windows Subsystem for Linux) oppure Docker. In un laboratorio scolastico con Windows e senza permessi di amministratore, questo aggiunge un ulteriore livello di complessità.

In conclusione, se utilizzato localmente Cassandra può risultare complesso e pesante, ma grazie alle soluzioni cloud diventa molto più accessibile e adatto anche a un contesto scolastico.

3 Integrazione web

L'integrazione di Cassandra con applicazioni web è generalmente buona, ma richiede una struttura più articolata rispetto ad altri database più semplici. Cassandra non è progettato per essere utilizzato direttamente dal lato client, ad esempio tramite JavaScript nel browser, per motivi di sicurezza e architettura. È sempre necessario un backend, cioè un server intermedio che gestisca le richieste tra il client e il database.

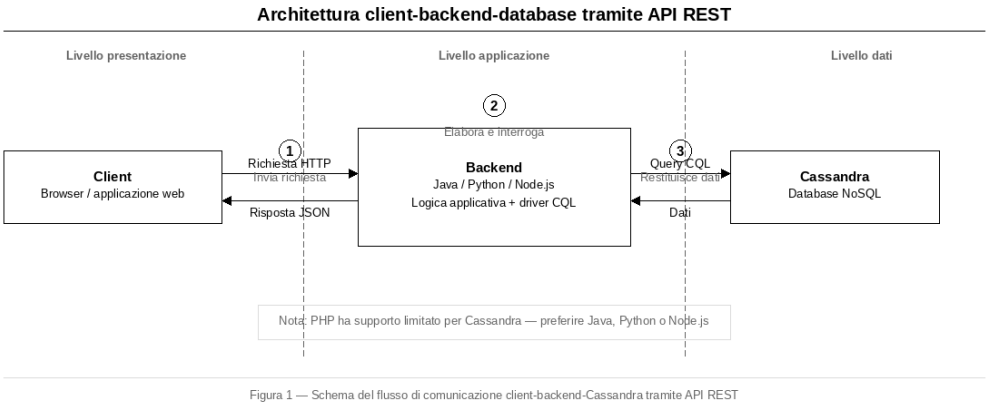

Figura 2: Schema del flusso di comunicazione client-backend-Cassandra tramite API REST

Cassandra si integra particolarmente bene con Java, Python e Node.js, per i quali esistono driver ufficiali che permettono di connettersi al database ed eseguire query in modo efficiente. Il legame con Java è particolarmente stretto, poiché Cassandra stesso è scritto in Java. Per chi ha già una conoscenza di base del linguaggio, questo rappresenta un vantaggio concreto: la sintassi del driver, la gestione delle eccezioni e la struttura del codice risulteranno familiari, abbassando la curva di apprendimento.

Per quanto riguarda PHP, è tecnicamente possibile utilizzare Cassandra, ma il supporto è meno sviluppato rispetto ad altri linguaggi. Le librerie disponibili possono richiedere configurazioni aggiuntive e risultare meno intuitive, rendendo questo approccio meno consigliabile in ambito scolastico.

L'aspetto più impegnativo per uno studente che si avvicina per la prima volta a Cassandra riguarda il modello di integrazione tramite API: il client invia una richiesta al backend, che interroga Cassandra tramite il driver e restituisce i dati. Chi non ha ancora familiarità con questo schema potrebbe trovarlo inizialmente complesso. Tuttavia, proprio per questo motivo, lavorare con Cassandra in un progetto web prepara in modo molto più solido al mondo del lavoro, dove questo tipo di architettura è ormai lo standard.

Cassandra richiede un investimento iniziale maggiore rispetto a soluzioni più semplici, ma chi già conosce Java parte con un vantaggio significativo. La complessità apparente diventa quindi un'opportunità formativa, introducendo concetti — come le API REST e l'architettura client-server — che risulteranno fondamentali in qualsiasi percorso professionale nel campo dell'informatica.

Consigliamo di introdurre Apache Cassandra nei laboratori non come sostituto di MySQL, ma come evoluzione. Sebbene richieda di tralasciare PHP a favore di Java o Node.js, questa scelta permette di insegnare la scalabilità orizzontale e il modello AP (Disponibilità e Tolleranza alla Partizione) del Teorema CAP. È un investimento didattico che prepara gli studenti a gestire i Big Data, usando gli stessi strumenti dei grandi leader del tech.

4 Percorso di studi: dal relazionale al distribuito

4.1 Fase 1: Le Basi con MySQL e PHP

In questa fase si costruiscono le fondamenta logiche della gestione dati.

- **Modello Relazionale:** Imparare a strutturare tabelle fisse e a normalizzare i dati per evitare ridondanze.

- **Integrità ACID:** Comprendere come garantire che ogni transazione sia sicura, atomica e coerente (fondamentale per banche e dati sensibili).
- **Interrogazione Standard:** Utilizzare SQL per eseguire query e JOIN complessi tra tabelle.
- **Sviluppo Monolitico:** Connettere il database a una pagina web tramite PHP, imparando il ciclo richiesta-risposta tipico delle applicazioni classiche.

4.2 Fase 2: Introduzione al Mondo NoSQL

Una volta compresi i limiti della scalabilità verticale di SQL, si esplorano nuove architetture.

- **Il Teorema CAP:** Analizzare perché in un sistema distribuito bisogna scegliere tra Coerenza (Consistency) e Disponibilità (Availability).
- **Il Modello BASE:** Studiare la "Coerenza Finale" (Eventual Consistency), dove i dati convergono verso uno stato comune nel tempo invece di essere sincronizzati istantaneamente.
- **Flessibilità dello Schema:** Capire i vantaggi di archiviare dati senza definire tabelle rigide in anticipo (Schema-less).

4.3 Fase 3: Specializzazione con Apache Cassandra

Dopo aver imparato le basi, si passa a gestire i Big Data e le scritture massicce.

- **Passaggio a Java/Node.js:** Abbandonare la struttura sincrona di PHP per utilizzare linguaggi con driver ufficiali più performanti per Cassandra.
- **Wide-Column Design:** Imparare a progettare tabelle basate sulle famiglie di colonne, dove ogni riga può avere un numero diverso di colonne.
- **Denormalizzazione:** Imparare a duplicare i dati strategicamente per ottimizzare le prestazioni di lettura, eliminando la necessità dei JOIN.
- **Scalabilità Orizzontale:** Comprendere come aggiungere nodi a un cluster per gestire traffico globale, proprio come fanno Netflix o Instagram.

4.4 Fase 4: Architetture Ibride e Progetto Finale

Il traguardo finale è saper scegliere lo strumento giusto per il compito giusto.

- **Sistemi Multi-Modello:** Integrare SQL per dati sensibili (account, pagamenti) e Cassandra per dati ad alto volume (log, cronologia, feed).

Questo percorso trasforma lo studente da un semplice utilizzatore di database a un progettista di sistemi scalabili, capace di passare dal rigore di MySQL alla potenza distribuita di Cassandra.

5 Documentazione e supporto

Uno dei punti di forza di Cassandra è la qualità della documentazione e il supporto disponibile online. Il database dispone di una documentazione ufficiale molto completa, che copre sia gli aspetti base che quelli più avanzati, come la gestione dei cluster, la replicazione dei dati e l'ottimizzazione delle prestazioni.

Oltre alla documentazione ufficiale, esistono numerosi tutorial, corsi online e guide pratiche che permettono agli studenti di apprendere gradualmente il funzionamento del database. Molti di questi materiali sono gratuiti e facilmente accessibili, il che rappresenta un grande vantaggio in ambito scolastico.

La community di Cassandra è abbastanza ampia e attiva, soprattutto perché il database è utilizzato anche da grandi aziende per la gestione di grandi quantità di dati. Questo significa che è possibile trovare facilmente soluzioni a problemi comuni tramite forum, siti di domande e risposte e piattaforme come GitHub, Stack Overflow, Reddit o Youtube.

Per uno studente, avere accesso a una buona documentazione e a una community attiva è fondamentale, perché permette di superare le difficoltà iniziali e di imparare in modo autonomo. In questo senso, Cassandra rappresenta una scelta valida anche dal punto di vista del supporto didattico.

6 Confronto con altri modelli NoSQL

Il panorama NoSQL comprende diverse famiglie di database, ciascuna progettata per risolvere problemi specifici. Per valutare Cassandra in modo oggettivo è utile confrontarlo con tre dei modelli più diffusi: MongoDB, Redis e Neo4j.

MongoDB è un database documentale: i dati vengono salvati come documenti JSON, senza uno schema fisso. È considerato il punto di ingresso più naturale al mondo NoSQL, proprio perché richiede poca configurazione iniziale e permette di iniziare a lavorare in pochi minuti. Tuttavia, questa semplicità ha un costo: MongoDB non è progettato per scalare orizzontalmente in modo nativo come Cassandra, e tende a soffrire di problemi di prestazioni quando i dati crescono molto e le scritture diventano frequenti. In ambito didattico è più facile da avvicinare, ma insegna meno concetti avanzati rispetto a Cassandra.

Redis è un database in-memory di tipo chiave-valore: tutti i dati vengono mantenuti nella RAM, il che lo rende estremamente veloce per operazioni semplici di lettura e scrittura. Non è però progettato per gestire grandi volumi di dati strutturati in modo persistente, e non dispone di un linguaggio di query strutturato paragonabile a CQL o SQL. Il suo utilizzo tipico è come cache davanti a un database principale, non come database primario. In un contesto scolastico, Redis è utile per capire il concetto di cache, ma non è adatto a sostituire un database relazionale o a insegnare la progettazione dei dati.

Neo4j è un database a grafo: i dati vengono rappresentati come nodi e relazioni, ed è particolarmente efficiente per query che coinvolgono connessioni complesse tra entità, come i social network o i sistemi di raccomandazione. Il suo linguaggio di interrogazione, Cypher, è molto diverso da SQL e richiede un cambio di paradigma significativo. Neo4j è uno strumento potente e formativo, ma il suo modello è difficilmente generalizzabile — funziona bene solo per problemi dove le relazioni tra i dati sono il focus principale.

7 Conclusioni

Cassandra si posiziona in modo diverso rispetto a tutti e tre. A differenza di MongoDB non nasce per la semplicità immediata, ma per la scalabilità e l'affidabilità su larga scala. A differenza di Redis, è un database persistente e strutturato, capace di gestire miliardi di record. A differenza di Neo4j, il suo modello è generalizzabile a moltissimi scenari reali, dagli e-commerce ai sistemi IoT, dai social network alle piattaforme di streaming.